

PQ-Shield: An Empirical Framework for Security–Performance Benchmarking of Post-Quantum Cryptography in Real-Time and Streaming AI Inference APIs

Nivitha K

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Manav Prakash

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Shashwat Kumar

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Yatharth Srivastava

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Abstract—AI inference APIs are protected almost entirely by classical key exchange and signatures, which a cryptographically relevant quantum computer (CRQC) could break retroactively: traffic recorded today could be decrypted later (harvest-now-decrypt-later, HNDL). We present PQ-Shield, an application-layer emulation of protected inference transactions that wraps a real FastAPI service in six configurations: Control, Classical-RSA (RSA-2048-OAEP with per-transaction key generation, a legacy worst case), Classical-ECDHE (X25519), Hybrid (ML-KEM-768 with ECDSA), Hybrid-KEX (X25519MLKEM768 with ECDSA), and Full PQC (ML-KEM-768 with ML-DSA-65). We measure them under concurrency, four payload shapes, byte-level network emulation, session reuse, and active and passive attacks, with streaming responses from a real Llama-3.2-3B model, and separate cryptographic from protocol overhead with a matched no-crypto control. Relative to that control, the cryptographic overhead at 10 connections is 1.3–2.7 ms per 92 ms transaction for Full PQC, Hybrid, and Classical-ECDHE, and Hybrid and Full PQC are statistically equivalent to X25519 within $\pm 5\%$ at 10 and 1,000 connections; in our emulation of a 100 ms, 5 Mbit/s path, ML-KEM adds about 5 ms of handshake time. Per-chunk post-quantum signing, not key exchange, is the expensive case: we treat per-chunk and hash-chained signing as endpoints of a hash chain checkpointed every k chunks and measure the trade-off between signature bytes and exposure, the tokens shown before verification (ML-DSA-65: 135,669 B at $k=1$, 3,309 B at $k=\infty$ per 200 tokens). Binding session context and a signed end record into the stream lets every tested drop, reorder, duplication, replay, and truncation attack be detected. Under classical key exchange a future CRQC could recover 5.8 bytes of plaintext per streamed token; under the modeled attack, ML-KEM-based key establishment exposes none. The ML-KEM bindings reproduce all tested NIST ACVP outputs byte for byte, and ML-DSA verification gives the expected verdict on all tested vectors.

Index Terms—post-quantum cryptography, ML-KEM, ML-

DSA, hybrid key exchange, harvest-now-decrypt-later, AI inference APIs, LLM streaming, stream authentication, benchmarking

I. Introduction

AI inference endpoints—hosted by commercial providers, on managed cloud platforms, or as custom FastAPI/Flask services—carry model inputs and outputs that can be sensitive: diagnostic features, financial records, and increasingly the full text of conversational and agentic sessions. Inference endpoints are already a recognised attack surface for interception, tampering, and privacy leakage [10], [11]. Their transport protection rests on RSA or elliptic-curve key exchange and signatures, all of which Shor’s algorithm breaks in polynomial time on a sufficiently large fault-tolerant quantum computer [21]. Such a machine does not yet exist, but the harvest-now-decrypt-later (HNDL) threat means an adversary can record encrypted traffic today and decrypt it once one does; data whose confidentiality must outlive the arrival of a CRQC is already exposed [13], [22].

NIST finalized its first post-quantum standards in August 2024: ML-KEM (FIPS 203) for key encapsulation [16] and ML-DSA (FIPS 204) for signatures [17]. Protocol-level migration is under way—the X25519MLKEM768 hybrid group is already negotiated by major browsers and content-delivery networks [20]—and post-quantum TLS has been benchmarked extensively [6], [14], [24], [25]. What is missing is an empirical account of what migration costs an AI inference workload specifically: small structured requests, inference times from milliseconds to seconds, and,

for large language models, responses that are streamed token by token and therefore cannot be signed as a whole before the first byte is sent.

This paper makes seven contributions. First, a concurrency benchmark (10–1,000 connections, five repetitions per cell) of six configurations on a real classifier workload, including an X25519 baseline representative of modern TLS key establishment, the deployed X25519MLKEM768 hybrid, and a matched no-crypto protocol control that separates cryptographic from protocol overhead, with server CPU and memory and repetition-level statistics (bootstrap confidence intervals, effect sizes, equivalence tests). Second, a payload-shape study across four AI workload profiles (163 B–8 kB). Third, network-emulation and session-reuse experiments that bound the cost of post-quantum message size and quantify how session reuse amortizes key establishment. Fourth, passive (HNDL) and active (tampering and stream-sequence) threat experiments integrated into the pipeline. Fifth, an adaptation of amortized stream signing [26]–[28] to token-by-token LLM responses on a real Llama-3.2-3B backend, where a post-quantum signature’s size (3,309 B for ML-DSA-65 versus ≈ 71 B for ECDSA P-256) makes per-chunk signing about forty times costlier in bytes. We show that naive per-chunk signing admits undetected reordering and truncation, and close both by binding a role label, the session context, and the chunk index into every signed message and ending each stream with a signed record of its length. Sixth, we treat per-chunk and hash-chained signing as the two endpoints of one design space—a hash chain checkpointed every k chunks—define exposure, the number of tokens a client displays before any signature covers them, and measure signature bytes, exposure, and mid-stream attack detection for $k \in \{1, 2, 5, 10, 20, \infty\}$ against an analytic model that the measurements match exactly. Seventh, ground-truth validation of the post-quantum bindings against NIST ACVP known-answer tests and of the streaming harness against the analytic model.

II. Background and Related Work

A. Post-Quantum Primitives

ML-KEM-768 (FIPS 203) is a module-lattice key-encapsulation mechanism at NIST security category 3, with public key, secret key, and ciphertext sizes of 1,184, 2,400, and 1,088 bytes and a 32-byte shared secret. ML-DSA-65 (FIPS 204) is a module-lattice signature scheme at the same category, with public key, secret key, and signature sizes of 1,952, 4,032, and 3,309 bytes. Several publications that post-date FIPS 204 still quote pre-standardization CRYSTALS-Dilithium3 values (a 3,293-byte signature and 4,000-byte secret key); all values in this paper are confirmed against FIPS 204 as finalized (Section VII). The X25519MLKEM768 hybrid [20] concatenates an ML-KEM-768 share with a 32-byte X25519

share and derives the session key from both shared secrets, so it stays secure unless both algorithms are broken.

B. Related Work

Table I synthesizes fifteen studies spanning heterogeneous-hardware PQC benchmarking, industry deployment, layered TLS 1.3 evaluation, migration surveys, 5G/6G integration, ML threat modeling, and HNDL economics. None combines AI-specific payload profiling, application-layer load sweeps, and integrated adversarial experiments, and none addresses streaming response signatures. Three further lines of work bear directly on our design.

Post-quantum TLS under network conditions: Paquin et al. [24] benchmarked post-quantum key exchange and signatures inside TLS 1.3 over emulated links (Linux netem), showing that on lossy or low-bandwidth paths message size, not computation, dominates handshake latency. Sikeridis et al. [25] reached a similar conclusion for post-quantum authentication, and Montenegro et al. [6] and Gómez-Cambronero et al. [14] extend the analysis to layered TLS and application-level load. These works measure the TLS handshake with generic payloads. PQ-Shield instead measures an application-layer protocol around AI inference payloads, including streaming responses, and adds controlled adversarial experiments. Our network emulation (Section V-H) is deliberately coarser than theirs—it shapes the byte stream rather than packets—and we use it to bound, not replace, their packet-level results.

Stream authentication: Signing a stream that is not available in full when transmission starts is a classical problem. Gennaro and Rohatgi [26] chain per-block hashes so that one signature authenticates an entire stream; Wong and Lam [27] amortize one signature over a flow with hash trees; TESLA and EMSS [28] trade immediate verification for low per-packet overhead. Our hash-chain strategy is an instance of this family, not a new construction, and inherits the same trade-off: integrity is confirmed only when a signature arrives. What is new is the setting. LLM token streams have a small, latency-critical first chunk and, with post-quantum signatures, a per-signature cost large enough that the choice of amortization decides whether streaming is affordable at all; we make that choice explicit and measurable through checkpointing (Section V-F).

Side channels in encrypted LLM streams: Weiss et al. [29] recovered the content of encrypted AI-assistant responses from the sizes of streamed packets (the token-length side channel). The attack is orthogonal to the key exchange and applies to every protected configuration here, but it sharpens one qualification: per-chunk and hash-chained signing emit one record per chunk and therefore expose chunk count and timing, which only buffer-and-sign hides.

TABLE I
Synthesis of related work and the gap each leaves open

Study	Focus	Limitation Addressed Here
Abbasi et al. [1]	Heterogeneous-hardware PQC benchmark	No AI workload or network stack
Demir et al. [2]	Industry TLS/telecom PQC deployment	No application-layer API metrics
Egbuagha & Ikwunna [3]	Protocol-level PQC readiness review	No experimental data
Pote & Bansode [4]	Micro-benchmark of core PQC operations	No web protocol context
Shaw [5]	Python X25519+ML-KEM hybrid library	No adversarial threat model
Montenegro et al. [6]	Post-quantum TLS 1.3 evaluation framework	Restricted to generic TLS traffic
Chhetri et al. [7]	Quantum-safe migration survey	Qualitative; no performance data
Faval et al. [8]	PQC in 5G core networks	Telecom-specific profiles
Kudaloor & Aijaz [9]	PQC for 6G wireless nodes	No REST/JSON API pipeline
Song et al. [10]	Privacy risks of robust ML models	Pre-dates practical PQC standards
Tidjon & Khomh [11]	Threat assessment for ML systems	Ignores transport-layer crypto
Ahmed et al. [12]	PQC support in crypto libraries	No end-to-end measurement
Blanco-Romero et al. [13]	Economic feasibility of HNDL	No application-layer context
Gómez-Cambronero et al. [14]	Layered TLS 1.3 handshake performance	Fixed key exchange; no signature sweep
Schemitt et al. [15]	PQC signatures on blockchains	Domain-restricted to blockchain

III. Threat Model

A. Harvest-Now-Decrypt-Later (HNDL)

A passive adversary records every cryptographic artifact on a protected connection—the key-establishment blob, every AES-256-GCM (nonce, ciphertext) pair, and any signatures—and stores them until a CRQC becomes available. We assume AES-256-GCM itself remains secure (Grover’s algorithm only halves its effective key length), so what differs between configurations is whether the recorded handshake lets the adversary recover the session key. Under Shor’s algorithm this holds for RSA-OAEP and X25519. For ML-KEM-768 and X25519MLKEM768 it does not under currently known quantum attacks, assuming a correct implementation. This is a statement about the modeled key-recovery attack on the recorded handshake, not a proof that the protocol resists every future attack; in particular, our handshake does not authenticate server keys with certificates (Section IV).

B. Active Man-in-the-Middle (MITM)

An active adversary on the wire intercepts a response and flips one byte, either inside the AEAD ciphertext or inside the signature, and we observe whether and where the client rejects it. A ciphertext flip should be rejected by the AEAD authentication-tag check before the payload reaches the application; a signature flip is caught only when the signature is verified. We log the detection layer and the time of the rejecting check itself, which separates configurations that merely detect tampering from those that detect it cheaply and early.

C. Streaming-Specific Threats

Two threats appear only when one connection streams many chunks. The first is a sequence-integrity attack, in which an on-path adversary rearranges validly protected records without forging any: it may drop a chunk, reorder two chunks, duplicate a chunk, substitute a validly signed chunk recorded from another session (cross-session replay), or truncate the stream by cutting it off before its last

chunks and its terminal record (forged termination). Each chunk’s AEAD tag and, under naive per-chunk signing, its signature remain valid, so detection requires binding every signed message to its role, its session, and its position, and authenticating the end of the stream. Our protocol binds a 3-byte role label, a 32-byte session context derived from the handshake identifier (one stream per handshake, so it also identifies the stream), and the chunk index into every signed message, and ends every stream with a signed record that commits to the number of chunks (Section V-B). We measure how early each strategy detects each attack.

The second is HNDL exposure that compounds with session length. A streamed session reuses one session key for every chunk, so a passive adversary who records the handshake captures the whole session at once: the longer the stream, the more ciphertext becomes retroactively readable the moment the key exchange is broken. We quantify this by relating the number of generated tokens to the volume a single future key recovery would expose. None of the evaluated configurations rotates keys within a session; mid-stream rekeying is left to future work.

IV. System Design

PQ-Shield is an application-layer emulation of protected inference transactions, not an implementation of TLS: it reproduces the cryptographic work of key establishment, payload encryption, and response signing around an inference call, without certificate authentication or TLS framing. Six processes wrap the same FastAPI inference server (Fig. 1): an unprotected control and five protected configurations (Table II) sharing a common ServerCryptoConfig/ClientCryptoConfig contract, so that only the asymmetric key-establishment and signature primitives differ. All protected configurations use the same AES-256-GCM layer keyed by HKDF-SHA256 over the raw shared secret. Every protected transaction follows the same five steps—handshake, session-key establishment,

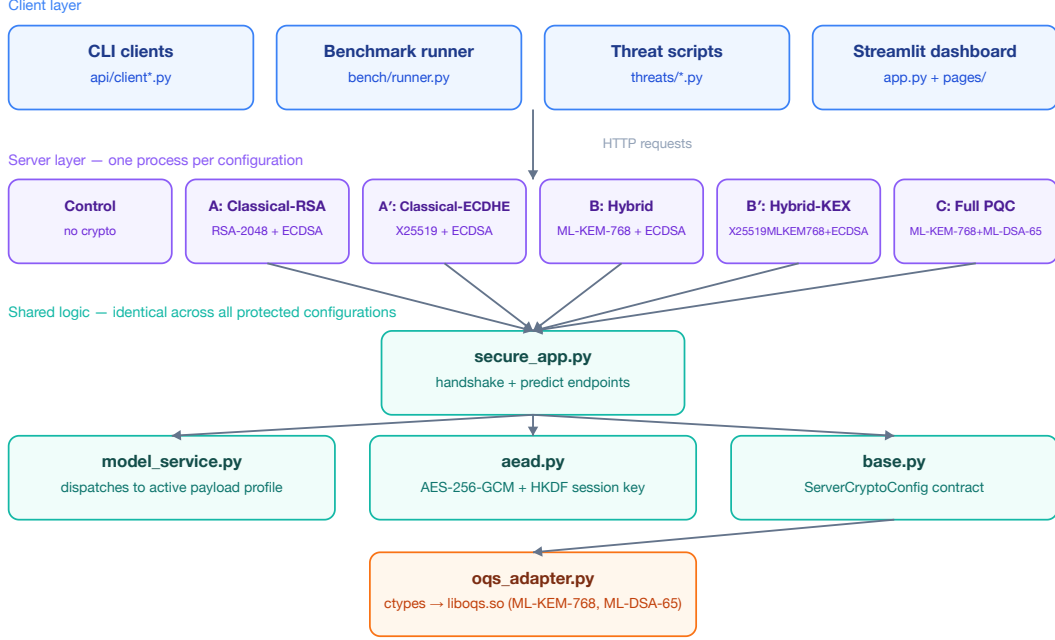


Fig. 1. PQ-Shield architecture. Six server processes wrap the same FastAPI inference service: an unprotected control and five protected configurations that share the handshake/predict endpoints, the AES-256-GCM + HKDF-SHA256 session layer, and one ServerCryptoConfig contract, so only the key-establishment and signature primitives differ. ML-KEM-768 and ML-DSA-65 come from a ctypes binding to liboqs; RSA, X25519, and ECDSA from OpenSSL via the Python cryptography package.

TABLE II
Configurations. ECDSA is over P-256; “PQ”: key exchange resists a CRQC. All protected configurations share AES-256-GCM with an HKDF-SHA256 session key.

Configuration	Key establishment	Signature	PQ
Control	none	none	—
A: Classical-RSA	RSA-2048-OAEP (new key per handshake)	ECDSA	no
A': Classical-ECDSA	ephemeral X25519	ECDSA	no
B: Hybrid	ML-KEM-768	ECDSA	yes
B': Hybrid-KEX	X25519MLKEM768	ECDSA	yes
C: Full PQ	ML-KEM-768	ML-DSA-65	yes

encrypt-and-send, decrypt, verify (Fig. 2)—with a fresh ephemeral key pair per transaction by default.

Configuration A' is an X25519-based classical baseline representative of the key-establishment primitive used in modern TLS deployments: TLS 1.3 removed RSA key transport [19], and ephemeral X25519 is the classical half of the X25519MLKEM768 group now negotiated on the web [20]. A' and B have identical signature code, so A' → B isolates exactly one change, X25519 to ML-KEM-768. Configuration A uses RSA-2048-OAEP key transport with per-transaction key generation; no production system generates RSA keys per handshake, so we include A as a legacy worst-case baseline that exposes the cost of RSA key generation under load, not as the classical baseline.

Configuration B pairs a post-quantum key exchange (ML-KEM-768 alone) with a classical signature. Configuration B' instead combines X25519 and ML-KEM-768 within the key exchange, following [20]: the server's share is the ML-KEM encapsulation key followed by an X25519 public key (1,216 B), the client's is the ML-KEM ciphertext followed by its X25519 public key (1,120 B), and the session key is derived from both shared secrets concatenated. With identical signature code, A' → B' is the migration step deployed on the web today, and B → B' the price of hedging against an ML-KEM break.

Streaming experiments use a real Llama-3.2-3B-Instruct model (Q4_K_M, llama.cpp with Metal). The concurrency, payload-shape, network, session-reuse, HNDL, and MITM experiments use the non-streaming endpoint with a RandomForest classifier (or the selected payload profile). One locally hosted LLM produces ≈40 tokens per second for a single stream, so a 1,000-connection LLM sweep would measure the model's queue rather than the cryptography.

V. Methodology

A. Environment and Process Isolation

All measurements were taken on one host (Apple M3 MacBook Air, 8 cores, 16 GB, macOS) on AC power with low-power mode disabled, on a single day. Each configuration's server runs in its own process, is health-checked, swept, and stopped before the next configuration

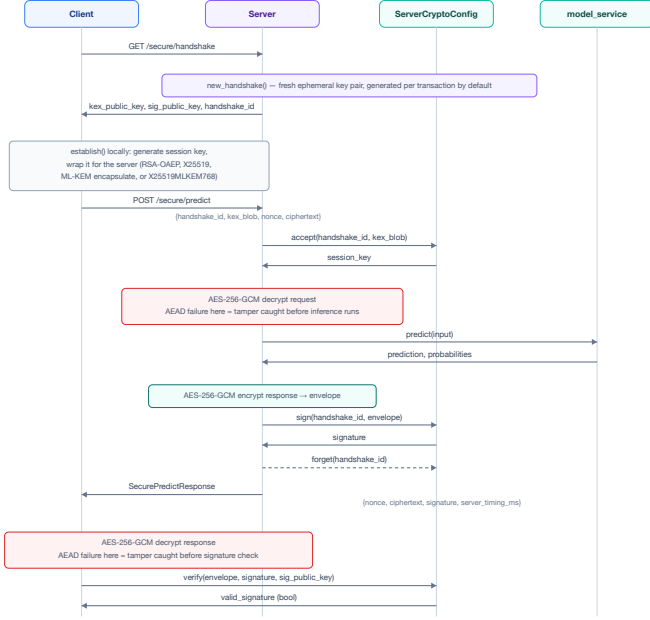


Fig. 2. One protected transaction. The client fetches fresh ephemeral keys, establishes the session key locally, and sends an encrypted request; the server decapsulates, decrypts (an AEAD failure here stops tampering before inference), infers, encrypts, and signs. The client decrypts first—an AEAD failure rejects a tampered response before any signature work—and then verifies the signature.

starts, so no two configurations share the CPU. All endpoints, including the control’s, run their request handlers in the server’s worker thread pool, so the control is not disadvantaged by blocking inference on the event loop. Every sweep records a run identifier, the host, and its parameters with each raw row, and the analysis uses each configuration’s most recent run.

B. Cryptographic Binding and Protocol Construction

ML-KEM-768 and ML-DSA-65 are accessed through a purpose-built ctypes binding against a locally compiled liboqs [23] rather than the higher-level liboqs-python wrapper. The binding exposes exactly the six C entry points used (key generation, encapsulation, and decapsulation for ML-KEM-768; key generation, signing, and verification for ML-DSA-65), with byte lengths asserted against the FIPS 203/204 constants at every call boundary, so a build or linkage error surfaces immediately rather than propagating malformed key material. liboqs is compiled with only these two algorithms, which keeps continuous re-verification of the binding (Section VII) cheap. RSA-OAEP, X25519, ECDSA P-256, and AES-256-GCM come from OpenSSL through the Python cryptography package.

The handshake orchestration, AES-256-GCM payload encryption (keyed by HKDF-SHA256 over the raw shared secret in every configuration, including Classical-RSA’s OAEP-transported key), and FastAPI plumbing are identical code paths across configurations, so differences be-

tween them can be attributed to the asymmetric primitives. By default a fresh ephemeral key pair is generated on every transaction—the conservative worst case; Section V-G measures the amortized case.

Stream message binding: For streaming, the same signature primitive is applied to strategy-specific messages, each beginning with a 3-byte role label and the session context $ctx = \text{SHA-256}(\text{"PQ-Shield-stream-v2"} \parallel 0x00 \parallel hid)$, where hid is the handshake identifier and i, n are 4-byte big-endian integers:

- buffer-and-sign: $\text{Sign}(\text{BUF} \parallel ctx \parallel \text{nonce} \parallel ct)$;
- per-chunk: $\text{Sign}(\text{CHK} \parallel ctx \parallel i \parallel \text{nonce} \parallel ct_i)$ for every chunk, then a terminal $\text{Sign}(\text{END} \parallel ctx \parallel n)$;
- hash-chain: $h_0 = \text{SHA-256}(\text{GEN} \parallel ctx)$,

$$h_i = \text{SHA-256}(h_{i-1} \parallel i \parallel \text{nonce}_i \parallel ct_i), \quad (1)$$

optional checkpoints $\text{Sign}(\text{CKP} \parallel ctx \parallel i \parallel h_i)$, and a terminal $\text{Sign}(\text{FIN} \parallel ctx \parallel n \parallel h_n)$.

The role label separates the five message types; ctx makes a record signed for one session fail verification in any other, even if a server reused one long-term signing key (ours are per-handshake, which also prevents it); the index binds position; and the terminal records commit to the stream length. The client requires the terminal record, checks its count against the chunks it accepted, and recomputes h_i itself rather than trusting transmitted chain hashes.

C. Concurrency Benchmark

Load is generated by an asynchronous client with a bounded worker pool, so the achieved concurrency matches the requested level as latency varies. Besides the control (one unprotected request per transaction) we run a matched-protocol control, Control-2RT, which makes the same two HTTP requests as a protected transaction—a handshake request and the inference request—with no cryptographic work. Protected minus Control-2RT is then the cryptographic overhead alone, and Control-2RT minus control the protocol overhead of the extra round trip. Each cell (configuration $\times$ concurrency $\times$ repetition) sends ten requests per connection (with a floor for low concurrency), and five repetitions are collected at 10, 100, and 1,000 connections. The first 5% of requests in each cell are discarded as warm-up. Latency is end-to-end (total_ms: handshake round trip plus protected request), the only quantity directly comparable to the control’s single request. A sampler records server CPU and resident memory every 0.25 s.

Statistical design: Requests within one repetition share a server process, its warm-up state, and its queue, so they are not independent; the repetition is our unit of analysis. We report the median, p95, failure rate, and the range of the five per-repetition medians. Differences between configurations are differences in medians with 95% confidence intervals from a two-stage bootstrap (1,000 draws: resample repetitions, then requests within each drawn repetition). Effect sizes are Cliff’s δ . Significance

uses the two-sided Mann–Whitney U test on per-repetition medians (five versus five, so the smallest attainable p is 0.008), Holm-corrected within each family of comparisons against one reference at one concurrency level. To support claims that two configurations perform the same, we test equivalence with two one-sided tests: a configuration is declared equivalent to a reference when the 90% bootstrap CI of the median difference lies within $\pm 5\%$ of the reference median. We call a difference practically significant only if its CI excludes a 5% band, not merely zero.

D. Payload-Shape Sensitivity

Four profiles are swept at 10 connections with all six configurations and five repetitions: `tabular_small` (a 100-tree RandomForest trained on the UCI handwritten-digits dataset, 345 B request / 93 B response), `image_cnn` (a fixed-weight NumPy convolutional network on a synthetic $32 \times 32 \times 3$ image, 4,138 B / 244 B), `embedding` (a synthetic 768-dimension vector generator, 202 B / 8,022 B), and `llm_completion` (a synthetic text generator, 163 B request). The synthetic profiles probe payload shape and size; they are not claims of representative model behaviour.

E. Streaming Signature Strategies

The streaming endpoint sends chunks as Server-Sent Events over one HTTP response; each event carries one strategy-specific record (nonce, ciphertext, and, depending on strategy, a signature and/or chain hash). The model’s blocking token generator is bridged into the server’s event loop through a worker thread so that generation does not stall other requests. Time to first token (TTFT) is measured at the client from issuing the request to receiving the first event. Each configuration $\times$ strategy $\times$ length cell (50 and 200 tokens, 5-token chunks) has ten repetitions; after each server start one discarded generation absorbs the model’s cold start, and the order of cells is shuffled with a fixed seed so that no strategy is always measured first.

F. Checkpointed Hash Chains and Exposure

The hash-chain strategy can additionally sign the running digest h_i every k chunks. The client verifies each checkpoint signature against its own recomputed h_i , so any dropped, reordered, or altered chunk before a checkpoint makes that checkpoint fail mid-stream. For an n -chunk response this costs $\lfloor n/k \rfloor + 1$ signatures (checkpoints plus the terminal one), and a decrypted chunk waits at most k chunks for coverage. We record two exposure measures per transaction: the largest number of chunks displayed but not yet covered by a verified signature (max unverified chunks), and the longest time any chunk waited for coverage (verification lag). Per-chunk signing is the $k=1$ endpoint (exposure one chunk) and plain hash-chain the $k=\infty$ endpoint (exposure the whole response). We sweep $k \in \{1, 2, 5, 10, 20, \infty\}$ for Hybrid (71 B signatures)

and Full PQC (3,309 B) on 200-token Llama responses in 5-token chunks (five repetitions), and at each k repeat the drop and reorder attacks, recording how much of the stream is delivered before detection.

G. Session Resumption

To measure the amortized case, a client may set `keep_session`: the server then caches the derived session key under the handshake identifier, later requests carry no key-establishment blob and reuse it, and the last request of the session ends it. Each of 10 concurrent clients reuses one key exchange for $N \in \{1, 10, 100\}$ requests (1,000 requests per cell, five repetitions), and we report end-to-end overhead against the control.

H. Network Emulation

All other experiments run over loopback, where moving an extra 1,088-byte ML-KEM ciphertext or 3,309-byte ML-DSA signature costs almost nothing. To expose the cost of message size we place a TCP proxy between load generator and server that adds a one-way propagation delay of $\text{RTT}/2$ in each direction and a per-direction bottleneck bandwidth shared by all connections; bytes are released in order, so HTTP semantics are unchanged. We evaluate four paths—loopback, metro (10 ms RTT, 100 Mbit/s), WAN (50 ms, 20 Mbit/s), and mobile (100 ms, 5 Mbit/s)—each with all six configurations at 10 connections and five repetitions. The proxy shapes the byte stream above TCP; it does not model slow start, the initial congestion window, loss, or fragmentation, all of which penalize large handshakes further [24], so the measured cost is a lower bound.

I. Threat Injection

For HNDL, the server reports, behind a debug header that does not alter ordinary traffic, the exact size of each cryptographic artifact it sends (1,000 transactions per configuration). We distinguish harvestable bytes, everything a passive adversary captures (key-establishment blob, AEAD records, signatures, chain hashes), from decryptable bytes, the encrypted application-payload records whose plaintext becomes recoverable once the session key is recovered. For a stream of n chunks,

$$D = \mathcal{W}_{\text{KEX}} \cdot \sum_{i=1}^n (|\text{nonce}_i| + |\text{ct}_i|), \quad (2)$$

where $\mathcal{W}_{\text{KEX}}$ is 1 if a CRQC can recover the session key from the recorded handshake (RSA-OAEP, X25519) and 0 otherwise (ML-KEM-768, X25519MLKEM768), $|\text{nonce}_i| = 12$ B and ct_i includes the 16-byte GCM tag; the recoverable plaintext is $D - 28n$ bytes. Signatures, public keys, and chain hashes are excluded from D : they are not secret. One session key protects the whole stream, and each 5-token chunk is one AEAD record, so D per token depends on the chunk size (framing) and on the tokenizer (plaintext). For streaming, response length is

varied with the configuration and strategy held fixed, so exposure can be attributed to length alone.

For MITM, a minimal forward proxy flips one bit in either the ciphertext or the signature field of each response (100 responses per configuration and target). The client decrypts before verifying, and detection time is the duration of the rejecting step itself—the AEAD tag check or the signature verification—so the two defence layers are directly comparable. Sequence attacks drop or swap a mid-stream chunk of a real streamed response and replay the sequence through the client’s incremental verification logic (ten trials per configuration, strategy, and attack).

J. Ground-Truth Validation

Two independent validations correspond to the two ways this paper’s results could be wrong. First, the primitive bindings are checked against official NIST Automated Cryptographic Validation Protocol (ACVP) known-answer tests [18] by re-deriving each vector’s output from its input and comparing byte for byte. Second, because no external dataset exists for streaming signature overhead, the streaming harness is checked against an analytic model: signature count and bytes follow deterministically from the number of chunks and the strategy (one signature for buffer-and-sign and plain hash-chain, one per chunk for per-chunk, $\lfloor n/k \rfloor + 1$ for checkpointed hash-chain), and signing time follows from the count and the independently measured per-signature cost.

K. Reproducibility and Artifact Availability

Every sweep records its run identifier, host, and parameters with each raw row. The software environment (OS and package versions, OpenSSL and liboqs revisions, the SHA-256 of the model files, and the repository commit) is captured automatically at run time and listed in Table XII. Code, configuration, result summaries, and the scripts that regenerate every figure and table are released at <https://github.com/Yuvicodes1/PQ-Shield-Benchmarking-Post-Quantum-Security-for-AI-APIs>.

VI. Results

A. Cost of Protection Under Concurrency

Table III and Fig. 3 summarize the sweep; differences are medians with 95% two-stage bootstrap CIs. At 10 connections the matched-protocol control (Control-2RT) costs +1.7 ms [−0.9, +4.0] over the control, so the extra round trip is negligible here, and the cryptographic overhead (protected minus Control-2RT) is +1.3 ms [−0.4, +3.9] for Full PQC, +2.3 ms [+0.3, +5.4] for Hybrid, +2.7 ms [+0.8, +5.2] for Classical-ECDHE, +5.4 ms [+3.1, +8.1] for Hybrid-KEX, and +76.8 ms [+67.6, +88.6] for Classical-RSA, against a 92 ms transaction. The ML-KEM configurations are statistically equivalent to the X25519 baseline within $\pm 5\%$ (Table IV; Hybrid −0.5% [−2.1, +1.7], Full PQC −1.5% [−3.0, +0.2], Hybrid-KEX +2.8% [+0.8, +4.9]), so replacing X25519 with ML-KEM-768 adds no practically significant latency in this

workload. Classical-RSA’s overhead sits almost entirely in the handshake, where the server generates a fresh RSA-2048 key pair (Fig. 4).

At 1,000 connections the four non-RSA protected configurations cost about twice the control (19.7–20.5 s against 9.4 s), but so does Control-2RT (21.3 s): measured against the matched protocol, their cryptographic overhead is −3.7 to −7.7%, with confidence intervals that include or approach zero. The doubling is therefore the second HTTP request per transaction, not the cryptography; Hybrid and Full PQC are again equivalent to Classical-ECDHE within $\pm 5\%$. Classical-RSA reaches 30.7 s (+44% over Control-2RT) and failures stay at 1.2–1.6%.

At 100 connections the per-repetition medians of the non-RSA protected configurations are bimodal—each repetition settles near either $\approx 1,000$ ms or $\approx 1,550$ ms (e.g. Classical-ECDHE 1,014–1,701 ms, Full PQC 988–1,604 ms)—while both controls are stable (Control-2RT 982–1,003 ms). The resulting overheads have wide intervals (e.g. Full PQC +341 ms [+38, +592] over Control-2RT), and no ML-KEM configuration can be shown equivalent to, or different from, Classical-ECDHE at this level. We attribute the regime switching to queueing in a single-host setup where client and server share the CPU, and do not rank configurations at 100 connections; only Classical-RSA (+140% over Control-2RT) is clearly separated.

RSA key generation is CPU-bound: In preliminary runs on a single-core host, Classical-RSA failed every request at 1,000 connections. On the 8-core host it failed only 1.5%, while consuming 3.5–4.8 cores of server CPU (Table V) against 0.6–0.9 for every other protected configuration. Per-handshake RSA key generation therefore collapses when cores run out and survives when they do not, at several times the compute of any alternative. The primitive benchmark explains why: RSA-2048 key generation runs at 16 operations/s (median 62.7 ms over 200 samples) against 98,357/s for ML-KEM-768, a 6,167× gap (Table XI, Fig. 12).

B. Server Resources

Apart from Classical-RSA, CPU and memory are flat across configurations: at 10 connections every non-RSA configuration, including the control, uses 0.92–0.94 cores and 231–241 MB of resident memory (Table V). Post-quantum key and signature sizes did not change memory use appreciably at this scale.

C. Payload Shape

Across the four profiles the cryptographic overhead is roughly constant in absolute terms (Fig. 5, Table VI): +4.0–9.4 ms for every non-RSA configuration and +71.6–110.2 ms for Classical-RSA. In relative terms it ranges from +4.5% for the 89 ms RandomForest to up to +240% for millisecond-scale synthetic profiles. The cost of protection is thus an additive term independent of payload size; whether it matters depends on the inference time

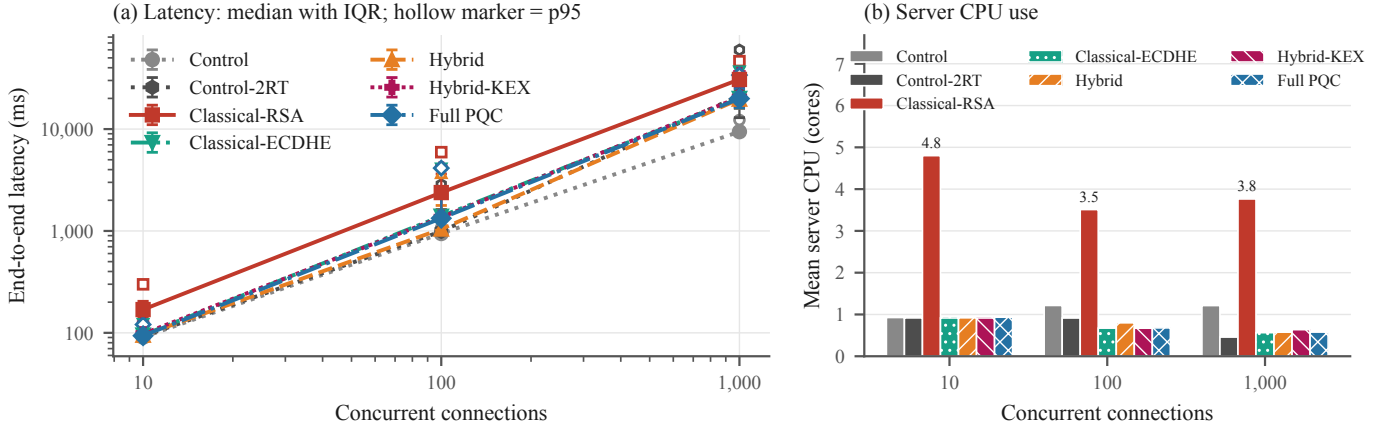


Fig. 3. Concurrency sweep. (a) End-to-end latency (median with interquartile range; hollow markers: p95) on log axes. The four non-RSA protected configurations overlap at every level and, at 1,000 connections, match the matched-protocol Control-2RT; Classical-RSA is consistently slowest. (b) Mean server CPU in cores (100% = one core): per-handshake RSA-2048 key generation occupies 3.5–4.8 cores, every other configuration about one core or less.

TABLE III

End-to-end latency (handshake + protected request) by configuration and concurrency, over successful requests after 5% warm-up trimming. “Rep. range”: spread of the five per-repetition medians. Δ : difference in medians (ms) with a 95% two-stage bootstrap CI (repetitions, then requests), versus Control and versus Control-2RT, the matched no-crypto protocol (making Δ_{2RT} the cryptographic overhead alone). δ : Cliff’s delta versus Control. p : two-sided Mann–Whitney U on per-repetition medians (5 vs. 5; smallest attainable 0.008), Holm-corrected within each concurrency level.

Conc.	Configuration	Median (ms)	Rep. range (ms)	p95 (ms)	Fail	Δ vs. Control	Δ_{2RT} (crypto)	δ	p_{Holm}
10	Control	90.7	89.1–92.3	117.0	0.0%	ref.	–	–	–
	Control-2RT	92.4	90.7–94.6	118.8	0.0%	+1.7 [−0.9, +4.0]	ref.	+0.10	0.222
	Classical-RSA	169.2	158.2–185.7	299.3	0.0%	+78.5 [+69.3, +91.3]	+76.8 [+67.6, +88.6]	+0.93	0.048
	Classical-ECDHE	95.2	93.7–96.8	121.6	0.0%	+4.4 [+2.2, +6.6]	+2.7 [+0.8, +5.2]	+0.19	0.048
	Hybrid	94.7	93.3–96.2	117.5	0.0%	+4.0 [+1.9, +6.7]	+2.3 [+0.3, +5.4]	+0.17	0.048
	Hybrid-KEX	97.8	96.7–99.8	123.4	0.0%	+7.1 [+4.5, +9.6]	+5.4 [+3.1, +8.1]	+0.27	0.048
	Full PQC	93.7	93.0–94.2	120.6	0.0%	+3.0 [+1.0, +5.2]	+1.3 [−0.4, +3.9]	+0.14	0.048
100	Control	942.9	922.8–976.1	1,084.1	0.0%	ref.	–	–	–
	Control-2RT	990.7	982.4–1,003.4	2,854.9	0.0%	+47.8 [+28.8, +66.2]	ref.	+0.24	0.048
	Classical-RSA	2,381.4	2,066.9–2,592.8	5,935.1	0.1%	+1.44 [+1.25, +1.59]s	+1.39 [+1.21, +1.54]s	+0.86	0.048
	Classical-ECDHE	1,396.2	1,014.0–1,701.4	4,081.1	0.1%	+453.3 [+120.0, +704.9]	+405.5 [+68.7, +654.9]	+0.50	0.048
	Hybrid	1,043.7	983.5–1,590.4	3,719.4	0.0%	+100.8 [+40.8, +422.6]	+53.0 [−5.1, +369.4]	+0.39	0.048
	Hybrid-KEX	1,361.7	1,115.1–1,638.5	4,179.9	0.2%	+418.9 [+199.1, +650.5]	+371.0 [+154.2, +603.2]	+0.47	0.048
	Full PQC	1,331.9	987.8–1,604.3	4,139.3	0.1%	+389.0 [+88.4, +645.2]	+341.2 [+38.3, +591.5]	+0.44	0.048
1,000	Control	9,428.0	9,402.4–9,586.0	12.27s	0.0%	ref.	–	–	–
	Control-2RT	21.31s	18.92s–23.89s	59.93s	0.2%	+11.88 [+10.52, +13.76]s	ref.	+0.77	0.048
	Classical-RSA	30.73s	29.94s–31.23s	46.42s	1.5%	+21.30 [+20.80, +21.62]s	+9.42 [+7.56, +10.88]s	+0.98	0.048
	Classical-ECDHE	20.00s	19.57s–20.37s	36.62s	1.5%	+10.57 [+10.23, +10.81]s	−1.31 [−3.15, +0.20]s	+0.92	0.048
	Hybrid	19.66s	19.09s–20.10s	34.23s	1.5%	+10.24 [+9.84, +10.55]s	−1.65 [−3.53, −0.17]s	+0.93	0.048
	Hybrid-KEX	20.52s	19.41s–22.53s	35.41s	1.2%	+11.09 [+10.28, +12.34]s	−0.79 [−2.63, +1.18]s	+0.95	0.048
	Full PQC	19.92s	19.68s–20.23s	33.88s	1.6%	+10.49 [+10.24, +10.56]s	−1.39 [−3.30, +0.13]s	+0.93	0.048

it is added to. Configurations using X25519 (Classical-ECDHE, Hybrid-KEX) are consistently 1.6–3.5 ms slower than those using ML-KEM alone, matching the primitive costs: server-side handshake work is 217.9 μ s for X25519 key generation plus exchange against 23.3 μ s for ML-KEM key generation plus decapsulation in our implementation.

D. Network Conditions

On emulated paths (Fig. 6) the handshake leg isolates the cost of key-exchange size. Relative to Classical-ECDHE, the ML-KEM configurations add $+4.6 \pm 0.5$ ms (Hybrid), $+4.7 \pm 0.3$ ms (Full PQC), and $+5.7 \pm 1.3$ ms

(Hybrid-KEX) on the mobile path, and at most 1.0 ms on the metro and WAN paths. End-to-end, these differences are smaller than the between-repetition variability of server queueing (standard deviation 15–26 ms on WAN and mobile), so they are not resolvable at the application level. Classical-RSA’s handshake is 61–81 ms slower than Classical-ECDHE’s on every path because of key generation, not bytes. Because the proxy does not model TCP slow start or loss, these are lower bounds.

TABLE IV

Equivalence of ML-KEM-based key establishment to the X25519 baseline (Classical-ECDHE): median end-to-end difference with its 90% two-stage bootstrap CI, as a percentage of the Classical-ECDHE median. “Equiv.”: the CI lies within $\pm 5\%$ (two one-sided tests at $\alpha = 0.05$).

Conc.	Configuration	Δ (%) [90% CI]	Equiv.
10	Hybrid	-0.5 [-2.1, +1.7]	yes
	Hybrid-KEX	+2.8 [+0.8, +4.9]	yes
	Full PQC	-1.5 [-3.0, +0.2]	yes
100	Hybrid	-25.2 [-43.2, +0.6]	no
	Hybrid-KEX	-2.5 [-27.9, +26.7]	no
	Full PQC	-4.6 [-35.7, +30.3]	no
1,000	Hybrid	-1.7 [-3.8, +0.6]	yes
	Hybrid-KEX	+2.6 [-1.3, +8.6]	no
	Full PQC	-0.4 [-1.9, +1.0]	yes

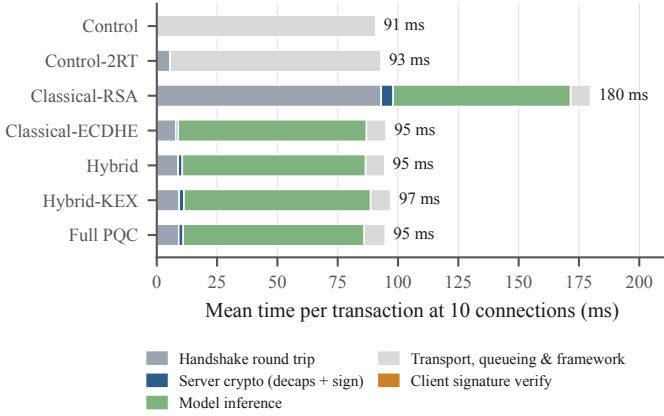


Fig. 4. Mean time per transaction at 10 connections, decomposed. Classical-RSA’s extra cost sits in the handshake (server-side RSA-2048 key generation); for the other configurations the handshake is ≈ 8 ms and model inference dominates.

E. Session Reuse

Fig. 7 reports overhead versus the control as one key exchange serves N requests. Classical-RSA’s $+101.2\%$ at $N=1$ disappears at $N=10$ ($-1.1 \pm 1.0\%$). For the other configurations the overhead falls from $+4.0$ – 7.0% at $N=1$ to between -0.2 and $+2.1\%$ at $N=100$; one intermediate point (Full PQC at $N=10$, $+9.0 \pm 8.4\%$) reflects a single outlying repetition. For deployments that reuse sessions, post-quantum key establishment cost at most about 2% end-to-end in this workload.

F. Streaming Signature Strategies

With the real Llama-3.2-3B backend (Fig. 8, Table VII), buffer-and-sign delays the first token to the end of generation (5.0–6.1 s for a 200-token response). Per-chunk and hash-chain signing both deliver the first token in 126–153 ms, 38 – $41\times$ sooner, in every configuration. The strategies differ in signature bytes: for Full PQC a 200-token response carries 135,669 B of ML-DSA-65 signatures under per-chunk signing (40 chunk signatures plus the signed end record) versus 3,309 B under hash-chain, a

TABLE V

Server-process resource use, sampled every 0.25 s during each cell and averaged over repetitions; RSS is mean resident memory (macOS, Apple M3, 8 logical CPUs; 100% = one core).

Conc.	Config.	CPU mean (%)	CPU max (%)	RSS (MB)	Req/s
10	Control	93	126	231	107.0
	Control-2RT	92	126	231	104.4
	Classical-RSA	480	699	238	52.4
	Classical-ECDHE	92	125	238	101.6
	Hybrid	92	126	239	102.1
	Hybrid-KEX	92	125	241	99.4
	Full PQC	94	127	241	102.1
100	Control	122	128	241	104.8
	Control-2RT	92	131	238	78.8
	Classical-RSA	351	792	247	35.8
	Classical-ECDHE	67	134	245	57.8
	Hybrid	80	130	247	68.1
	Hybrid-KEX	67	132	249	56.7
	Full PQC	68	133	250	58.3
1,000	Control	121	129	269	102.5
	Control-2RT	46	138	214	38.4
	Classical-RSA	376	800	283	32.9
	Classical-ECDHE	56	137	280	46.1
	Hybrid	58	139	286	47.7
	Hybrid-KEX	64	142	294	45.8
	Full PQC	58	139	290	47.7

$41\times$ difference; with ECDSA the corresponding figures are $\approx 2,910$ B and 71 B. Within each configuration the strategy order was shuffled, so these comparisons are unbiased; across configurations, however, generation slowed from ≈ 40 to ≈ 33 tokens/s over the 18-minute run (thermal throttling of the fanless host), so we do not compare streaming times between configurations.

G. Bounded Exposure with Checkpointed Hash Chains

Fig. 9 and Table VIII sweep the checkpoint interval k . Signature bytes follow the analytic model $(\lfloor n/k \rfloor + 1) \cdot |\sigma|$ exactly: for Full PQC 135,669 B (41 signatures) at $k=1$, 29,781 B at $k=5$, 9,927 B at $k=20$, and 3,309 B at $k=\infty$; for Hybrid $\approx 2,913$ B to 71 B. Exposure grows as bytes fall: the client displays at most 5, 25, 100, and 200 tokens before a signature covers them, with maximum verification lags of 0 ms, about 0.6 s, 2.6 s, and 5.6 s (Full PQC). The attack experiments show the security effect directly. Without checkpoints, hash-chain detects every mid-stream drop and reorder but only at the end of the stream. With checkpoints every drop and reorder is caught mid-stream (100%), after 50–51% of the stream at $k \leq 2$ (the attack position itself), 59–60% at $k=5$, 72% at $k=10$, and 97–98% at $k=20$. Cross-session replay is caught at the replayed chunk for every k , because a foreign chunk fails AEAD decryption under this session’s key. Detection depends only on where checkpoints fall, so Hybrid and Full PQC give identical curves.

H. Harvest-Now-Decrypt-Later Exposure

A passive adversary captures 209–433 B per classical transaction and 1,265–4,503 B per post-quantum transac-

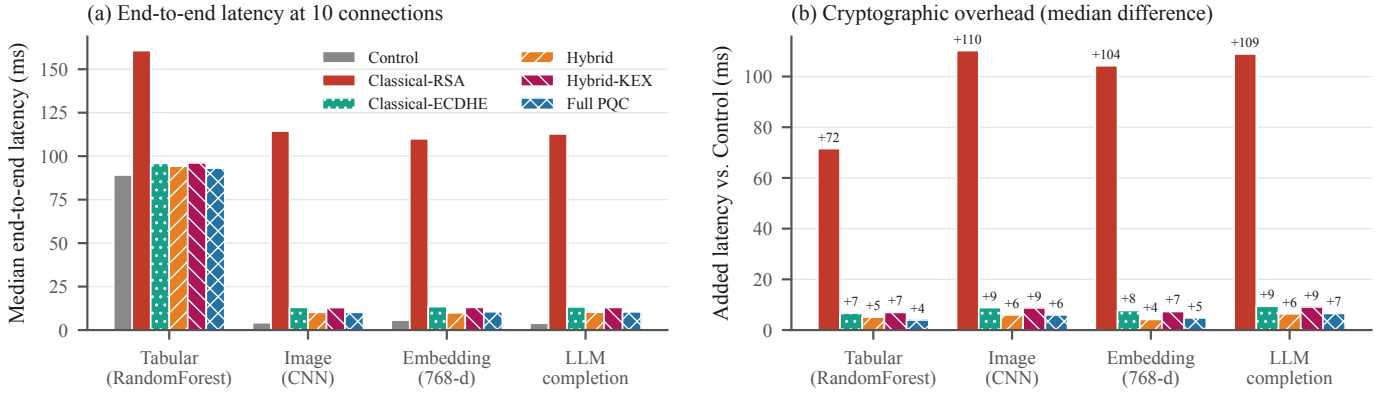


Fig. 5. Payload-shape sensitivity at 10 connections. (a) Median end-to-end latency per profile. (b) Added latency versus the control: roughly constant in milliseconds across profiles, so the cryptographic cost is additive and independent of payload size.

TABLE VI

Payload-shape sensitivity at 10 connections (5 repetitions per cell). Request/response sizes are plaintext medians; each cell gives median end-to-end latency (ms) and the added latency versus Control, with Mann–Whitney U significance (* $p < 0.05$).

Profile	Req. (B)	Resp. (B)	Control	Classical-RSA	Classical-ECDHE	Hybrid	Hybrid-KEX	Full PQC
tabular_small	348	90	89.2	160.7 (+71.6*)	95.9 (+6.7*)	94.3 (+5.1*)	96.1 (+7.0*)	93.1 (+4.0*)
image_cnn	4,138	245	4.2	114.4 (+110.2*)	13.1 (+8.9*)	10.2 (+6.0*)	12.9 (+8.7*)	10.2 (+6.0*)
embedding	202	8,017	5.7	110.0 (+104.2*)	13.5 (+7.7*)	9.9 (+4.2*)	13.0 (+7.3*)	10.5 (+4.8*)
llm_completion	157	4,048	3.9	112.8 (+108.9*)	13.3 (+9.4*)	10.3 (+6.4*)	13.0 (+9.1*)	10.5 (+6.6*)

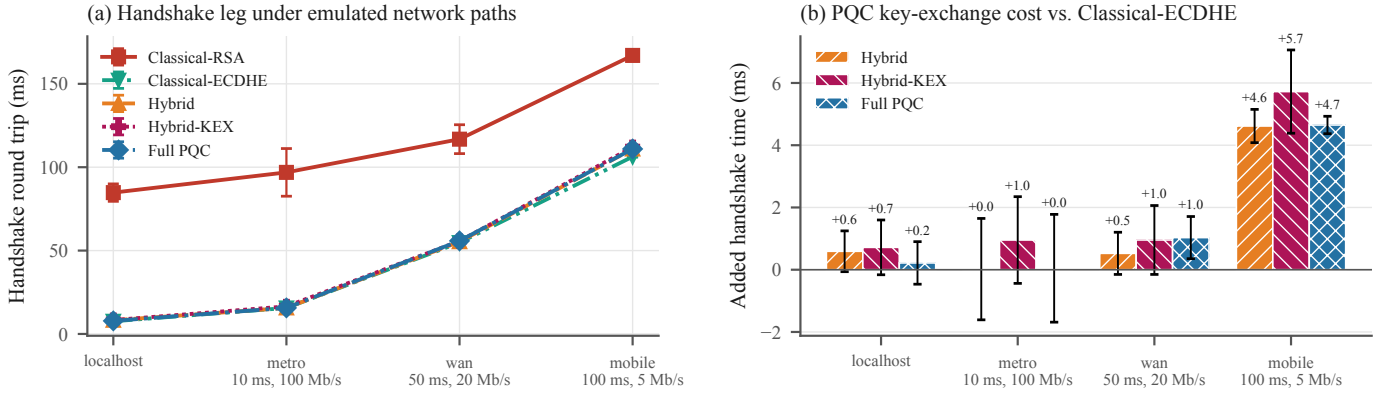


Fig. 6. Emulated network paths (10 connections; mean of five per-repetition medians, error bars one standard deviation). (a) Handshake round trip. (b) Added handshake time of the ML-KEM configurations relative to Classical-ECDHE: about 5 ms on the 5 Mbit/s mobile path, at most 1 ms elsewhere.

tion (Fig. 10). Only the classical captures—RSA-OAEP and X25519 alike—become decryptable once a CRQC exists. Under streaming (Fig. 11) the decryptable volume D grows linearly with response length for both classical key exchanges, at 11.4 bytes per generated token ($R^2=0.9997$, 50–585 tokens; the model ends longer requests early), of which 5.8 bytes per token are recoverable plaintext and the rest is per-chunk AEAD framing. Under the modeled attack, all three ML-KEM configurations have zero decryptable bytes at every length.

I. Active Tampering

Every tampered response was rejected in every configuration (Table IX). A flipped ciphertext bit is caught by

the AES-GCM tag in 0.007–0.008 ms (median), before the signature is examined; a flipped signature bit is caught by verification in 0.059 ms (ML-DSA-65) to 0.083 ms (ECDSA P-256). All five sequence attacks were detected in every trial, in every configuration and strategy. Per-chunk signing detects drop, reorder, duplicate, and cross-session replay at the affected chunk (100% mid-stream). Hash-chain detects replay immediately (the foreign chunk fails AEAD) and drop, reorder, and duplicate at the end unless checkpoints are used (Section VI-G). Truncation is detected by both strategies only at the end, through the missing signed terminal record; without that record, as in a naive design, a truncated per-chunk stream would verify

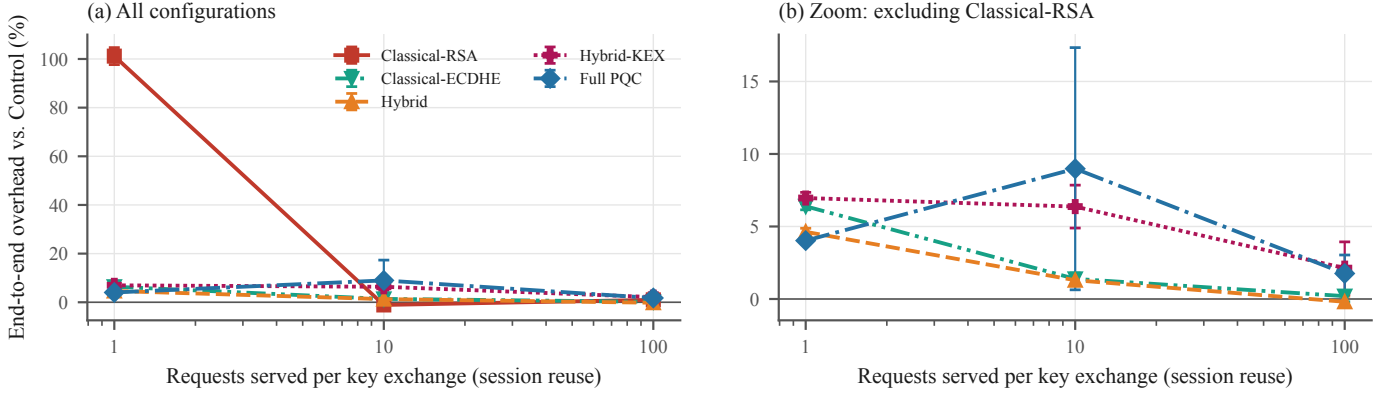


Fig. 7. Session reuse at 10 connections (mean $\pm$ one standard deviation over five repetitions). (a) All configurations: Classical-RSA’s overhead vanishes once a key exchange serves ten requests. (b) Zoom without Classical-RSA: post-quantum overhead falls to 0–2% at 100 requests per key exchange.

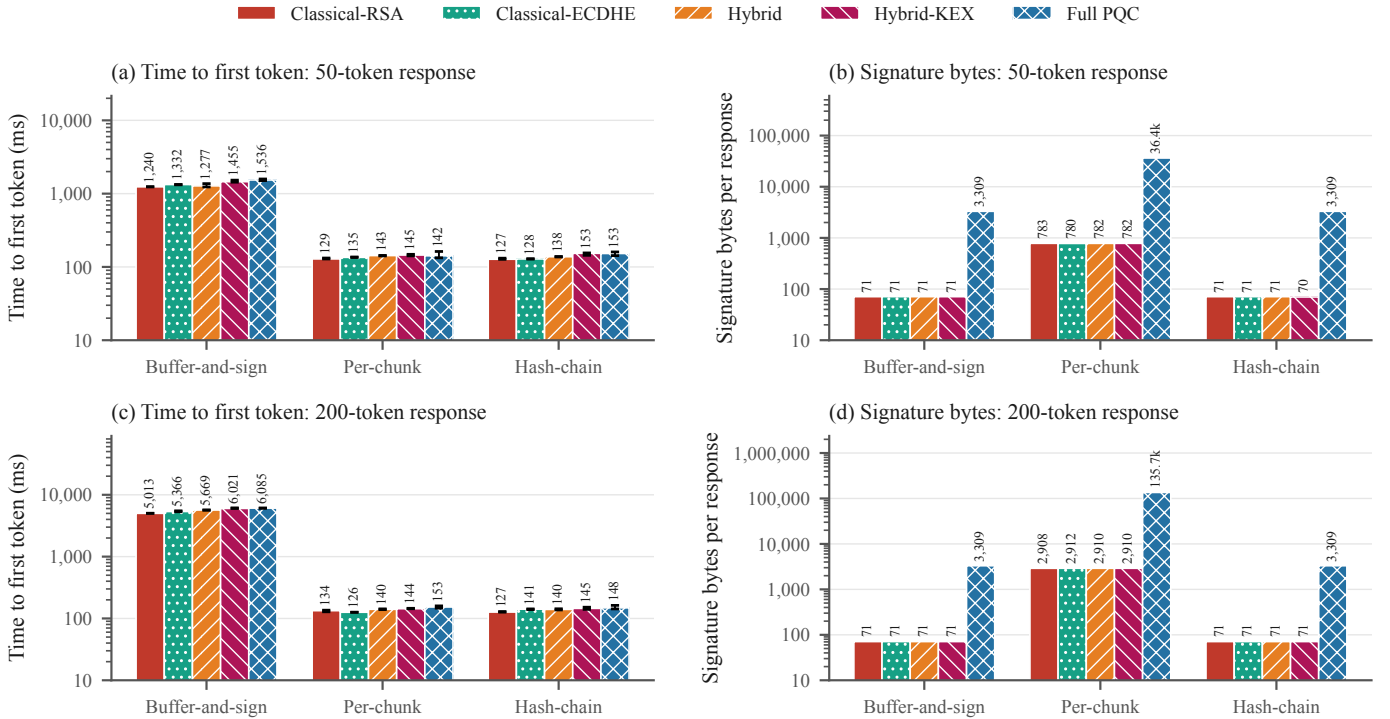


Fig. 8. Streaming signature strategies on Llama-3.2-3B (5-token chunks; median of ten repetitions, error bars interquartile range). (a, c) Time to first token: per-chunk and hash-chain deliver the first token 38–41 $\times$ sooner than buffer-and-sign. (b, d) Signature bytes per response: per-chunk ML-DSA-65 signing costs 41 $\times$ the bytes of hash-chain.

completely.

VII. Ground-Truth Validation

All 75 tested NIST ACVP known-answer vectors pass (Table X): our ML-KEM-768 key generation, encapsulation, and decapsulation reproduce the published outputs byte for byte, and ML-DSA-65 verification returns the expected verdict on every tested vector. This checks the correctness of our bindings on these vectors; it is not ACVP certification and does not cover ML-DSA key generation or signing. Byte-exact ML-DSA-65 key and signature reproduction is not possible through liboqs’s

public API, which exposes neither derandomized key generation nor deterministic signing.

Signature-byte predictions of the analytic model matched the harness’s measurements exactly for the three strategies and, in the checkpoint sweep, for all twelve configuration–interval cells (for example 41 ML-DSA-65 signatures, 135,669 B, at $k=1$). Timing validation surfaced a genuine effect rather than a confirmation: ECDSA P-256’s first signing call in a fresh process took 7.7–9.3 ms against a 37.8–42.3 μ s warm-loop mean, a 211–283 $\times$ gap attributable to lazy OpenSSL backend initialization,

TABLE VII

Streaming signature strategies on a real Llama-3.2-3B-Instruct backend (10 repetitions per cell; median, with interquartile range for TTFT). Signing and verification times are summed over the whole stream.

Tokens	Config.	Strategy	TTFT (ms)	Total (ms)	Sigs	Sig. bytes	Sign (ms)	Verify (ms)
50	Classical-RSA	Buffer-and-sign	1,239.6 [1,239–1,241]	1,240	1	71	0.07	0.07
		Per-chunk	128.6 [128–132]	1,252	10	783	0.43	0.73
		Hash-chain	127.3 [127–132]	1,242	1	71	0.08	0.07
	Classical-ECDHE	Buffer-and-sign	1,331.8 [1,319–1,335]	1,332	1	71	0.07	0.08
		Per-chunk	135.1 [134–137]	1,382	10	780	0.43	0.73
		Hash-chain	128.5 [128–129]	1,280	1	71	0.08	0.07
	Hybrid	Buffer-and-sign	1,277.0 [1,234–1,370]	1,277	1	71	0.07	0.08
		Per-chunk	142.9 [141–144]	1,446	10	782	0.43	0.72
		Hash-chain	137.8 [137–139]	1,400	1	71	0.08	0.07
	Hybrid-KEX	Buffer-and-sign	1,455.5 [1,434–1,529]	1,456	1	71	0.34	0.37
		Per-chunk	145.3 [142–149]	1,478	10	782	2.39	4.02
		Hash-chain	153.3 [144–155]	1,530	1	70	0.25	0.31
	Full PQC	Buffer-and-sign	1,535.8 [1,505–1,585]	1,537	1	3,309	0.61	0.28
		Per-chunk	141.9 [133–162]	1,431	10	36,399	4.40	2.53
		Hash-chain	152.5 [143–160]	1,525	1	3,309	0.53	0.21
200	Classical-RSA	Buffer-and-sign	5,013.0 [5,007–5,035]	5,013	1	71	0.07	0.09
		Per-chunk	133.6 [128–136]	4,943	40	2,908	1.47	2.59
		Hash-chain	126.8 [126–129]	4,948	1	71	0.07	0.07
	Classical-ECDHE	Buffer-and-sign	5,365.7 [5,323–5,504]	5,366	1	71	0.08	0.09
		Per-chunk	126.0 [126–126]	5,015	40	2,912	1.49	2.57
		Hash-chain	141.2 [139–143]	5,730	1	71	0.08	0.08
	Hybrid	Buffer-and-sign	5,669.1 [5,634–5,688]	5,669	1	71	0.07	0.09
		Per-chunk	140.5 [139–143]	5,833	40	2,910	1.58	2.65
		Hash-chain	140.0 [137–143]	5,657	1	71	0.07	0.08
	Hybrid-KEX	Buffer-and-sign	6,021.3 [5,999–6,143]	6,022	1	71	0.30	0.29
		Per-chunk	143.7 [144–146]	5,880	40	2,910	1.72	2.77
		Hash-chain	145.5 [140–152]	6,057	1	71	0.30	0.34
	Full PQC	Buffer-and-sign	6,085.0 [6,065–6,106]	6,086	1	3,309	0.57	0.24
		Per-chunk	153.1 [147–160]	6,193	40	135,669	22.81	10.88
		Hash-chain	148.5 [143–163]	6,158	1	3,309	0.41	0.25

TABLE VIII

Checkpointed hash-chain signing on Llama-3.2-3B (200-token responses, 5-token chunks): a signature every k chunks plus the terminal one. Exposure = most tokens a client displays before a signature covers them (median over repetitions).

Config.	k	Sigs	Bytes	Exp. (tok)	Lag (ms)	Sign+ver. (ms)
Hybrid	1	41	2,913	5	0	27.30
	2	21	1,489	10	172	13.68
	5	9	639	25	557	5.91
	10	5	355	50	1,210	0.69
	20	3	212	100	3,451	1.01
	∞	1	71	200	5,298	0.65
Full PQC	1	41	135,669	5	0	5.67
	2	21	69,489	10	146	17.95
	5	9	29,781	25	626	7.52
	10	5	16,545	50	1,232	0.76
	20	3	9,927	100	2,647	1.87
	∞	1	3,309	200	5,570	0.57

while ML-DSA-65 through our liboqs binding showed no such penalty. Correcting for this cold-start effect reduced the two largest timing discrepancies from $281 \times / 297 \times$ to $1.33 \times / 1.41 \times$; a smaller residual ($3\text{--}9 \times$) remains only partially explained. Byte-overhead results are unaffected.

VIII. Discussion and Decision Framework

We construct a weighted composite score,

$$\text{composite} = w_{\text{sec}} \cdot S - w_{\text{perf}} \cdot O, \quad (3)$$

where O is the end-to-end latency overhead versus the control and S an explicit ordinal security score: 0.0 for Classical-RSA and Classical-ECDHE (no quantum resistance on either axis), 0.8 for Hybrid and Hybrid-KEX (post-quantum key exchange prevents the modeled HNDL key recovery; the ECDSA signature remains classically vulnerable, a materially smaller risk because signatures protect real-time integrity rather than the confidentiality of harvested traffic), and 1.0 for Full PQC. Rather than collapse this judgment into one number, Table XIII reports the matrix at three weightings. Full PQC scores highest in most cells because its overhead matches Hybrid’s while its security score is higher; the cells at 100 connections inherit the queueing bimodality of Section VI-A and should not be used to rank configurations.

Hypotheses: H1 (bounded post-quantum overhead) is supported: Hybrid and Full PQC are equivalent to Classical-ECDHE within $\pm 5\%$ at 10 and 1,000 connections, their cryptographic overhead over the matched protocol is 1.3–2.3 ms at 10 connections, and in the byte-level emulation they add at most 5.7 ms of handshake

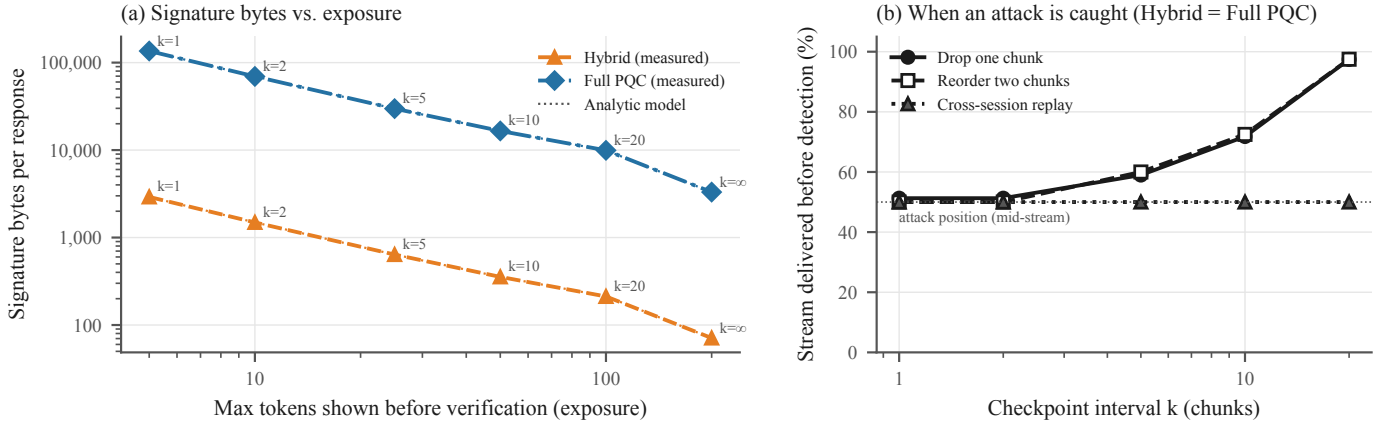


Fig. 9. Checkpointed hash chains on Llama-3.2-3B (200-token responses, 5-token chunks). (a) Signature bytes against exposure (most tokens displayed before verification); the measurements lie on the analytic model. (b) Share of the stream delivered before a mid-stream drop, reorder, or cross-session replay is detected; the dotted line marks the attack position.

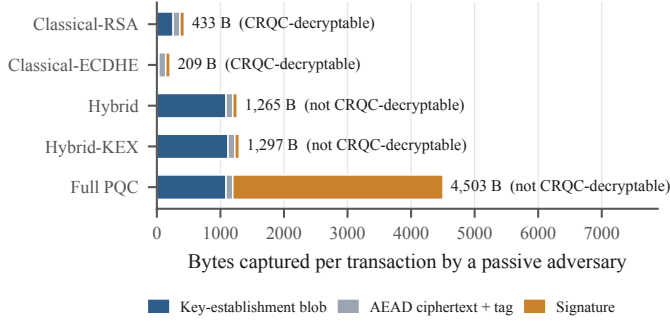


Fig. 10. Bytes a passive adversary captures per transaction, by component. Larger post-quantum captures are not decryptable by a future CRQC; the smaller classical ones are.

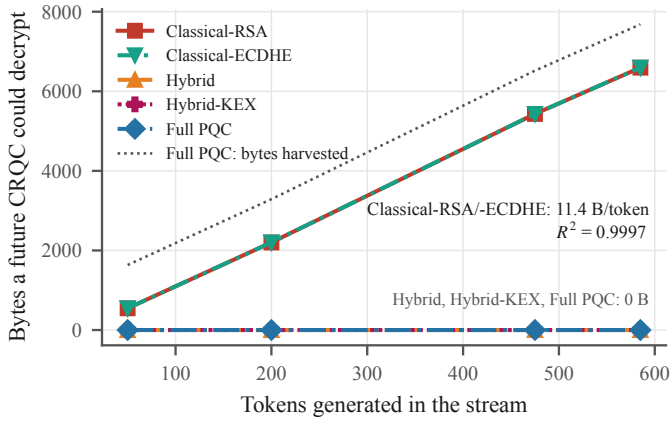


Fig. 11. Streaming HNDL exposure on Llama-3.2-3B. Under classical key exchange (RSA or X25519) the bytes a future CRQC could decrypt grow by 11.4 per generated token; under every ML-KEM configuration they stay at zero although more bytes are harvested.

time; at 100 connections queueing bimodality prevents a conclusion. H2 (Hybrid captures Full PQC's HNDL benefit at lower cost) is supported on HNDL, where both have zero decryptable bytes under the modeled attack,

but not on cost: Full PQC is no slower than Hybrid in any non-streaming experiment and costs more only in signature bytes when many signatures are sent. H3 (categorically different decryptability) is demonstrated: classical exposure grows linearly with streamed tokens, ML-KEM-based exposure stays at zero. H4 (ML-DSA-65 verification not slower than ECDSA P-256) is confirmed: 28,202 versus 21,602 verifications/s ($1.31\times$).

IX. Recommendations

Key establishment: Within the evaluated threat model, migrating from X25519 to ML-KEM-768 removes the modeled HNDL exposure at no practically significant latency cost in our workload (equivalent within $\pm 5\%$ at 10 and 1,000 connections), about 5 ms of handshake time on a slow emulated link, and 0–2% once sessions are reused. Where hedging against an ML-KEM weakness matters, the X25519MLKEM768 hybrid cost about 3 ms more than ML-KEM alone at 10 connections in our implementation. Per-handshake RSA key generation should be avoided: it occupied 4–5 CPU cores at modest load and fails when cores run out.

Signatures: ML-DSA-65 verification is faster than ECDSA P-256, so Full PQC added no practically significant latency for single-shot responses. Its 3,309 B signature matters only where many signatures are sent, which in practice means streaming.

Streaming: Use hash-chained signing with checkpoints, bind session context and a signed end-of-stream record (Section V-B), and choose k from the exposure the application can tolerate: $k=\infty$ (one signature; the response is unverified until it ends) for low-risk display, small k where a user must not act on unverified text. With ML-DSA-65, $k=5$ bounds exposure to 25 tokens for 29,781 B per 200-token response, against 135,669 B for per-chunk signing. Buffer-and-sign is justified only when chunk-count and timing metadata must be hidden [29], at a 38–41 $\times$ time-to-first-token penalty.

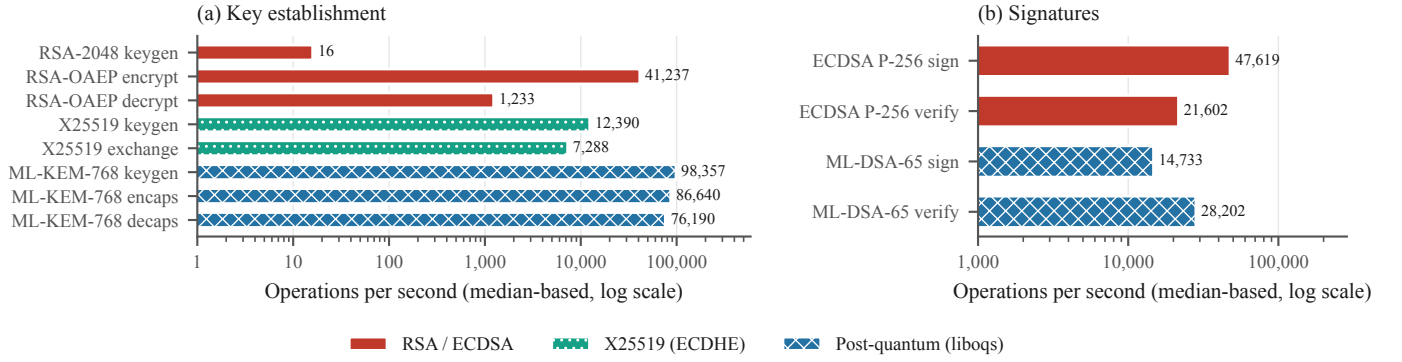


Fig. 12. Primitive throughput (single process, warm loop, median-based). (a) Key establishment, log scale: RSA-2048 key generation is $6,167\times$ slower than ML-KEM-768 key generation, and ML-KEM-768 is also cheaper than X25519. (b) Signatures: ML-DSA-65 verification is $1.31\times$ faster than ECDSA P-256 despite a $\approx 47\times$ larger signature.

X. Limitations

- Single host. All measurements come from one machine (Apple M3, 8 cores, 16 GB). The load generator and server share its CPU, which matters most at 1,000 connections. Relative comparisons between configurations are the intended result; absolute latencies are host-specific.
- Application-layer protocol, not TLS. PQ-Shield’s handshake is a purpose-built protocol. Its default fresh key exchange per transaction isolates the asymmetric primitive and is a worst case; Section V-G measures the amortized case, but not TLS’s own resumption mechanisms (PSK, 0-RTT). Server keys are not certificate-authenticated, so it is not a substitute for authenticated TLS.
- Network emulation is byte-level. The proxy models propagation delay and bandwidth but not TCP congestion control, loss, or MTU fragmentation. Results are a lower bound on post-quantum network cost; packet-level studies [24], [25] remain the reference for lossy links.
- Classical-RSA is a worst case. Its per-handshake RSA-2048 key generation does not reflect production practice; its failure under load is a property of that design, contrasted with the realistic Classical-ECDHE baseline.
- Hybrid-KEX measured separately. Configuration B’ was added after the main concurrency sweep; its concurrency results come from a supplementary run on the same host and day. Its streaming behaviour was measured with the others; its signature path is identical to Hybrid’s.
- Workloads. The `image_cnn` profile uses an untrained fixed-weight network and the `embedding/llm_completion` profiles are synthetic; they probe payload shape and size, not model behaviour. Streaming uses one quantized 3B-parameter model, which may end a stream before the requested length, so streaming HNDL is reported against tokens actu-

ally generated.

- Queueing at 100 connections. At 100 connections the protected configurations switch between two latency regimes across repetitions while both controls do not; with client and server on one host we cannot fully explain this, and we do not rank the non-RSA configurations at that level. A separate load-generator host would resolve it.
- Thermal drift in streaming. Over the 18-minute streaming sweep, generation slowed from ≈ 40 to ≈ 33 tokens/s on the fanless host. Strategy order was shuffled within each configuration, so strategy comparisons are unbiased, but streaming times are not compared across configurations; signature bytes are unaffected.
- Repetitions. Five repetitions per cell bound the precision of between-run comparisons; with repetition medians as the unit, the smallest attainable p is 0.008, and confidence intervals at 100 connections are wide because of queueing bimodality.
- Timing-validation residual. A $3\text{--}9\times$ gap between predicted and measured streaming signing time for some strategy-configuration pairs remains only partially explained; signature-byte predictions match exactly.

XI. Conclusion and Future Work

Measured on a real inference service through an application-layer emulation, the answers to the practical questions are as follows; each holds under our threat model and on our single host.

Strongest security: Full PQC (ML-KEM-768 with ML-DSA-65) is the only evaluated configuration whose key establishment and signatures both resist known quantum attacks. Hybrid-KEX offers the most conservative key establishment, since X25519MLKEM768 stays secure unless both of its components are broken, but its ECDSA signatures remain classically vulnerable. Combining X25519MLKEM768 with ML-DSA-65 would dominate both; we did not measure it.

Best performance: Among realistic configurations there is no single fastest one: Full PQC, Hybrid, and Classical-ECDHE are statistically equivalent within $\pm 5\%$ at 10 and 1,000 connections, with cryptographic overheads of 1.3–2.7 ms per 92 ms transaction; Hybrid-KEX adds about 5 ms. Classical-RSA with per-transaction key generation is clearly the slowest. When many signatures are sent—per-chunk streaming or frequent checkpoints—the ECDSA-signed configurations are cheapest in bytes, by a factor of about 47 per signature.

Best balance: For non-streaming inference APIs, and for streaming with hash-chained signing and few checkpoints, Full PQC offers the best balance: the strongest security at no practically significant latency cost, and it scores highest in the weighted decision matrix in 7 of 9 cells. When every chunk must be signed or checkpoints are frequent, Hybrid (or Hybrid-KEX) is the better balance: at $k=1$ a 200-token response carries 2,913 B of ECDSA signatures instead of 135,669 B of ML-DSA-65.

What the answer depends on: Streaming: the signing schedule, not the key exchange, determines the post-quantum cost, through the number of signatures sent. Session reuse: reusing one key exchange for 100 requests reduced every non-RSA configuration’s overhead to 0–2%, making the key-exchange choice nearly free. Threat model: if only HNDL confidentiality matters, Hybrid suffices; quantum-resistant signatures or long-term non-repudiation require Full PQC; hedging against a break of ML-KEM itself favours Hybrid-KEX. Network: slow links add a few milliseconds of handshake time for ML-KEM, more under real packet loss than our byte-level emulation shows.

Future work includes a separate load-generator host and packet-level network measurements, a TLS-native implementation with certificate authentication and resumption, mid-stream rekeying to bound HNDL exposure per session, the X25519MLKEM768 with ML-DSA-65 combination, and a trained, larger-scale vision workload.

Acknowledgment

The authors thank the faculty and technical staff of the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, for the resources and infrastructure used in this study, and the open-source cryptographic and machine-learning communities, whose tools and standards supported its implementation, validation, and evaluation.

References

- [1] M. Abbasi, F. Cardoso, P. Váz, J. Silva, and P. Martins, “A practical performance benchmark of post-quantum cryptography across heterogeneous computing environments,” *Cryptography*, vol. 9, no. 2, art. 32, 2025, doi: 10.3390/cryptography9020032.
- [2] E. D. Demir, B. Bilgin, and M. C. Onbasli, “Performance analysis and industry deployment of post-quantum cryptography algorithms,” *arXiv:2503.12952*, 2025.
- [3] O. Egbugha and E. Ikwunna, “Post-quantum cryptography in practice: A literature review of protocol-level transitions and readiness,” *IACR Cryptology ePrint Archive*, Paper 2025/1668, 2025.
- [4] P. Pote and R. Bansode, “Performance evaluation of post-quantum cryptography: A comprehensive framework for experimental analysis,” *J. Inf. Syst. Eng. Manage.*, vol. 10, no. 9s, pp. 548–556, 2025, doi: 10.52783/jisem.v10i9s.1253.
- [5] A. Shaw, “quantum-safe: Bridging the post-quantum production gap with a hybrid-by-default Python cryptography library,” *arXiv:2605.17061*, 2026.
- [6] J. A. Montenegro, R. Rios, and J. Lopez-Cerezo, “A performance evaluation framework for post-quantum TLS,” *Future Gener. Comput. Syst.*, vol. 175, art. 108062, 2026, doi: 10.1016/j.future.2025.108062.
- [7] G. Chhetri, S. Somvanshi, P. Hebli, S. Brotee, and S. Das, “Post-quantum cryptography and quantum-safe security: A comprehensive survey,” *arXiv:2510.10436*, 2025.
- [8] R. A. Faval, R. Moreira, and F. de Oliveira Silva, “Empowering mobile networks security resilience by using post-quantum cryptography,” *arXiv:2603.28626*, 2026.
- [9] A. Kudaloor and A. Aijaz, “Toward quantum-safe 6G: Experimental evaluation of post-quantum cryptography techniques,” *IEEE Commun. Standards Mag.*, accepted, 2026, *arXiv:2605.06881*.
- [10] L. Song, R. Shokri, and P. Mittal, “Privacy risks of securing machine learning models against adversarial examples,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 241–257.
- [11] L. N. Tidjon and F. Khomh, “Threat assessment in machine learning based systems,” *arXiv:2207.00091*, 2022.
- [12] N. Ahmed, L. Zhang, and A. Gangopadhyay, “A survey of post-quantum cryptography support in cryptographic libraries,” in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2025, *arXiv:2508.16078*.
- [13] J. Blanco-Romero, F. Almenares Mendoza, C. García Rubio, C. Campo, and D. Díaz Sánchez, “On the practical feasibility of harvest-now, decrypt-later attacks,” *arXiv:2603.01091*, 2026.
- [14] D. Gómez-Cambronero, D. Munteanu, and A. I. González-Tablas, “Layered performance analysis of TLS 1.3 handshakes: Classical, hybrid, and pure post-quantum key exchange,” *arXiv:2603.11006*, 2026.
- [15] A. G. Schemitt, H. F. da Silva, R. C. Lunardi, D. Kreutz, R. B. Mansilha, and A. F. Zorzo, “Assessing the impact of post-quantum digital signature algorithms on blockchains,” in *Proc. IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, 2025, *arXiv:2510.09271*.
- [16] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” *FIPS PUB 203*, Aug. 2024, doi: 10.6028/NIST.FIPS.203.
- [17] National Institute of Standards and Technology, “Module-lattice-based digital signature standard,” *FIPS PUB 204*, Aug. 2024, doi: 10.6028/NIST.FIPS.204.
- [18] National Institute of Standards and Technology, “Automated Cryptographic Validation Protocol (ACVP) server and test vectors,” 2024. [Online]. Available: <https://github.com/usnistgov/ACVP-Server>
- [19] E. Rescorla, “The Transport Layer Security (TLS) protocol version 1.3,” *IETF RFC 8446*, Aug. 2018.
- [20] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, “Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3,” *IETF Internet-Draft draft-ietf-tls-ecdhe-mlkem-05*, May 2026.
- [21] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [22] M. Mosca, “Cybersecurity in an era with quantum computers: Will we be ready?” *IEEE Secur. Privacy*, vol. 16, no. 5, pp. 38–41, 2018.
- [23] D. Stebila and M. Mosca, “Post-quantum key exchange for the Internet and the Open Quantum Safe project,” in *Selected Areas in Cryptography – SAC 2016*, ser. LNCS, vol. 10532, Springer, 2017, pp. 14–37.

- [24] C. Paquin, D. Stebila, and G. Tamvada, “Benchmarking post-quantum cryptography in TLS,” in *Post-Quantum Cryptography – PQCrypto 2020*, ser. LNCS, vol. 12100. Springer, 2020, pp. 72–91.
- [25] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, “Post-quantum authentication in TLS 1.3: A performance study,” in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2020.
- [26] R. Gennaro and P. Rohatgi, “How to sign digital streams,” in *Advances in Cryptology – CRYPTO ’97*, ser. LNCS, vol. 1294. Springer, 1997, pp. 180–197.
- [27] C. K. Wong and S. S. Lam, “Digital signatures for flows and multicasts,” *IEEE/ACM Trans. Netw.*, vol. 7, no. 4, pp. 502–513, 1999.
- [28] A. Perrig, R. Canetti, J. D. Tygar, and D. Song, “Efficient authentication and signing of multicast streams over lossy channels,” in *Proc. IEEE Symp. Secur. Privacy*, 2000, pp. 56–73.
- [29] R. Weiss, D. Ayzenshteyn, G. Amit, and Y. Mirsky, “What was your prompt? A remote keylogging attack on AI assistants,” in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 3367–3384.

TABLE IX

Active tampering. (a) One flipped bit per response; detection time is the rejecting check alone. (b) Mid-stream sequence attacks on 60-token streams: share of trials detected at all, and before the stream completed (buffer-and-sign has no intermediate chunks to attack).

(a) Single-response tampering					
Config.	Target	n	Detected	Median detection (ms)	
Classical-RSA	ciphertext	100	100%	0.007	
Classical-RSA	signature	100	100%	0.063	
Classical-ECDHE	ciphertext	100	100%	0.008	
Classical-ECDHE	signature	100	100%	0.083	
Hybrid	ciphertext	100	100%	0.008	
Hybrid	signature	100	100%	0.083	
Hybrid-KEX	ciphertext	100	100%	0.008	
Hybrid-KEX	signature	100	100%	0.082	
Full PQC	ciphertext	100	100%	0.008	
Full PQC	signature	100	100%	0.059	
(b) Streaming sequence attacks					
Config.	Strategy / attack	n	Detected	Mid-stream	
Classical-RSA	Per-chunk / drop	10	100%	100%	
Classical-RSA	Per-chunk / duplicate	10	100%	100%	
Classical-RSA	Per-chunk / reorder	10	100%	100%	
Classical-RSA	Per-chunk / replay	10	100%	100%	
Classical-RSA	Per-chunk / truncate	10	100%	0%	
Classical-RSA	Hash-chain / drop	10	100%	0%	
Classical-RSA	Hash-chain / duplicate	10	100%	0%	
Classical-RSA	Hash-chain / reorder	10	100%	0%	
Classical-RSA	Hash-chain / replay	10	100%	100%	
Classical-RSA	Hash-chain / truncate	10	100%	0%	
Classical-ECDHE	Per-chunk / drop	10	100%	100%	
Classical-ECDHE	Per-chunk / duplicate	10	100%	100%	
Classical-ECDHE	Per-chunk / reorder	10	100%	100%	
Classical-ECDHE	Per-chunk / replay	10	100%	100%	
Classical-ECDHE	Per-chunk / truncate	10	100%	0%	
Classical-ECDHE	Hash-chain / drop	10	100%	0%	
Classical-ECDHE	Hash-chain / duplicate	10	100%	0%	
Classical-ECDHE	Hash-chain / reorder	10	100%	0%	
Classical-ECDHE	Hash-chain / replay	10	100%	100%	
Classical-ECDHE	Hash-chain / truncate	10	100%	0%	
Hybrid	Per-chunk / drop	10	100%	100%	
Hybrid	Per-chunk / duplicate	10	100%	100%	
Hybrid	Per-chunk / reorder	10	100%	100%	
Hybrid	Per-chunk / replay	10	100%	100%	
Hybrid	Per-chunk / truncate	10	100%	0%	
Hybrid	Hash-chain / drop	10	100%	0%	
Hybrid	Hash-chain / duplicate	10	100%	0%	
Hybrid	Hash-chain / reorder	10	100%	0%	
Hybrid	Hash-chain / replay	10	100%	100%	
Hybrid	Hash-chain / truncate	10	100%	0%	
Hybrid-KEX	Per-chunk / drop	10	100%	100%	
Hybrid-KEX	Per-chunk / duplicate	10	100%	100%	
Hybrid-KEX	Per-chunk / reorder	10	100%	100%	
Hybrid-KEX	Per-chunk / replay	10	100%	100%	
Hybrid-KEX	Per-chunk / truncate	10	100%	0%	
Hybrid-KEX	Hash-chain / drop	10	100%	0%	
Hybrid-KEX	Hash-chain / duplicate	10	100%	0%	
Hybrid-KEX	Hash-chain / reorder	10	100%	0%	
Hybrid-KEX	Hash-chain / replay	10	100%	100%	
Hybrid-KEX	Hash-chain / truncate	10	100%	0%	
Full PQC	Per-chunk / drop	10	100%	100%	
Full PQC	Per-chunk / duplicate	10	100%	100%	
Full PQC	Per-chunk / reorder	10	100%	100%	
Full PQC	Per-chunk / replay	10	100%	100%	
Full PQC	Per-chunk / truncate	10	100%	0%	
Full PQC	Hash-chain / drop	10	100%	0%	
Full PQC	Hash-chain / duplicate	10	100%	0%	
Full PQC	Hash-chain / reorder	10	100%	0%	
Full PQC	Hash-chain / replay	10	100%	100%	
Full PQC	Hash-chain / truncate	10	100%	0%	

TABLE X
NIST ACVP known-answer-test results (75/75)

Test	Result	Basis
ML-KEM-768 keygen (derand., Alg. 16)	25/25	byte-exact
ML-KEM-768 encapsulation (Alg. 17)	25/25	byte-exact
ML-KEM-768 decapsulation (incl. 5 implicit-rej.)	10/10	byte-exact
ML-DSA-65 signature verification (Alg. 3)	15/15	correct verdict

TABLE XI
Primitive-level micro-benchmark (single process, warm loop). n = timed iterations.

Algorithm	Op.	n	Median (μ s)	p99 (μ s)	ops/s
ML-KEM-768	KeyGen	2000	10.2	12.5	98,357
ML-KEM-768	Encaps	2000	11.5	14.7	86,640
ML-KEM-768	Decaps	2000	13.1	15.0	76,190
X25519	KeyGen	2000	80.7	93.5	12,390
X25519	Exchange	2000	137.2	150.1	7,288
RSA-2048	KeyGen	200	62,698.0	257,378.1	16
RSA-2048 OAEP	Encrypt	2000	24.3	27.3	41,237
RSA-2048 OAEP	Decrypt	2000	811.3	1,730.7	1,233
ML-DSA-65	KeyGen	2000	38.0	42.3	26,316
ML-DSA-65	Sign	2000	67.9	246.1	14,733
ML-DSA-65	Verify	2000	35.5	40.8	28,202
ECDSA P-256	KeyGen	2000	15.5	18.5	64,687
ECDSA P-256	Sign	2000	21.0	23.7	47,619
ECDSA P-256	Verify	2000	46.3	54.2	21,602

TABLE XII
Software and hardware environment, captured automatically at run time.

Component	Version
Host	Apple M3, 8 cores, 16 GB
OS	macOS 26.6.2
Python	3.13.5
FastAPI / Uvicorn / httpx	0.141.1 / 0.52.4 / 0.28.1
cryptography / OpenSSL	50.0.0 / 4.0.1
liboqs (commit)	8979276ad1eb
NumPy / SciPy / pandas	2.5.2 / 1.18.1 / 3.0.5
scikit-learn	1.9.0
llama-cpp-python (Metal)	0.3.35
LLM file	Llama-3.2-3B-Instruct-Q4_K_M.gguf
LLM SHA-256 (prefix)	6c1a2b4116103267
Classifier SHA-256 (prefix)	68b46ef7e1efce32

TABLE XIII

Weighted composite score $w_{sec} \cdot S - w_{perf} \cdot O$ (higher is better), where S is the security score (Classical 0, Hybrid 0.8, Full PQC 1.0) and O the end-to-end latency overhead versus Control. Bold: best at that weighting and concurrency; fail: every request failed; n/a: not measured.

Config.	Security ($w_{sec}=0.8$)			Balanced ($w_{sec}=0.5$)			Performance ($w_{sec}=0.2$)		
	10	100	1,000	10	100	1,000	10	100	1,000
Classical-RSA	-0.17	-0.31	-0.45	-0.43	-0.76	-1.13	-0.69	-1.22	-1.81
Classical-ECDHE	-0.01	-0.10	-0.22	-0.02	-0.24	-0.56	-0.04	-0.38	-0.90
Hybrid	+0.63	+0.62	+0.42	+0.38	+0.35	-0.14	+0.12	+0.07	-0.71
Hybrid-KEX	+0.62	+0.55	+0.40	+0.36	+0.18	-0.19	+0.10	-0.20	-0.78
Full PQC	+0.79	+0.72	+0.58	+0.48	+0.29	-0.06	+0.17	-0.13	-0.69

TABLE XIV
Recommendation summary

Scenario	Recommendation
Any inference API	Key exchange: ML-KEM-768, or X25519MLKEM768 to hedge against an ML-KEM break
Non-streaming responses	ECDSA P-256 or ML-DSA-65 (no latency difference)
Streaming, display-only	Hash-chain, $k = \infty$
Streaming, actionable output	Hash-chain with small k (e.g. $k = 5$: 25-token exposure)
Any streaming	Bind session context; sign an end-of-stream record
Long-term non-repudiation	ML-DSA-65 signatures (Full PQC)
Metadata-sensitive streaming	Buffer-and-sign
Legacy RSA key transport	Migrate away